

Скриптове и автоматизация - Batch и Bash

Увод

Скрипт е текстов файл, в който са записани команди за автоматично изпълнение. Вместо да въвеждаме една и съща поредица от действия многократно, можем да ги запишем в скрипт и да ги стартираме с една команда.

Какво е автоматизация?

- **Автоматизация** означава компютърът да изпълнява повтарящи се задачи вместо човека.
- Така се спестява време, намаляват се грешките и се осигурява еднакъв резултат при всяко изпълнение.
- Скриптовете се използват за копиране на файлове, архивиране, стартиране на програми, проверка на системата и много други задачи.

Batch

Batch е език за скриптове в Windows командния интерпретатор `cmd.exe`.
Файловете обикновено са с разширение `.bat` или `.cmd`.

Bash

Bash е команден интерпретатор и скриптов език, използван главно в Linux и Unix системи.
Скриптовете често са с разширение `.sh`.

1. Batch и Bash накратко

Характеристика	Batch	Bash
Операционна система	Windows	Linux / Unix / macOS / Git Bash / WSL
Типичен файл	<code>script.bat</code>	<code>script.sh</code>
Интерпретатор	<code>cmd.exe</code>	<code>/bin/bash</code>
Начин на запис на променливи	<code>%name%</code>	<code>\$name</code>
Коментар	<code>REM</code> или <code>::</code>	<code>#</code>
Сила на езика	Подходящ за по-прости автоматизации	По-гъвкав и по-мощен за системни задачи

Важно: И двата езика служат за автоматизация, но синтаксисът им е различен. Ако един скрипт работи в `Batch`, това не означава, че ще работи и в `Bash` без промени.

2. Как се създава и изпълнява скрипт

Batch в Windows

1. Отвори текстов редактор като Notepad или Visual Studio Code.
2. Запиши файла с разширение `.bat`, например `hello.bat`.
3. Стартирай го с двоен клик или през `cmd`.

```
@echo off
echo Zdravei, sveta!
pause
```

Bash в Linux

1. Създай файл, например `hello.sh`.
2. Добави първи ред `#!/bin/bash`.
3. Дай право за изпълнение с `chmod +x hello.sh`.
4. Стартирай с `./hello.sh`.

```
#!/bin/bash
echo "Zdravei, sveta!"
```

3. Подробен синтаксис на Batch

3.1. Основна структура на Batch файл

```
@echo off
REM Това е коментар
echo Nachalo na skripta
pause
```

- `@echo off` спира показването на самите команди по време на изпълнение.
- Символът `@` пред първата команда скрива точно тази команда.
- `echo` извежда текст на екрана.
- `pause` спира изпълнението и изчаква натискане на клавиш.

3.2. Коментари в Batch

```
REM Това е коментар  
:: Това също често се използва като коментар
```

Коментарите служат за обяснения в кода. Те не се изпълняват като команди.

3.3. Променливи в Batch

```
@echo off  
set ime=Ivan  
echo Zdravei, %ime%
```

- `set ime=Ivan` създава променлива с име `ime`.
- Стойността се използва чрез `%ime%`.
- Не се оставят интервали около знака `=`, освен ако не искаме те да станат част от стойността.

3.4. Четене на вход от потребителя

```
@echo off  
set /p name=Введи име:  
echo Zdravei, %name%
```

`set /p` показва съобщение и записва въведения текст в променлива.

3.5. Аритметика в Batch

```
@echo off
set a=5
set b=3
set /a sum=a+b
echo Sborat e %sum%
```

`set /a` позволява аритметични операции: събиране, изваждане, умножение, деление и остатък.

3.6. Условни конструкции в Batch

```
@echo off
set /p num=Vuvedi chislo:
if %num% GEQ 10 (
    echo Chisloto e po-golyamo ili ravno na 10
) else (
    echo Chisloto e po-malko ot 10
)
```

- `if` проверява условие.
- `GEQ` означава "greater or equal" - по-голямо или равно.
- Други оператори са `EQU`, `NEQ`, `LSS`, `LEQ`, `GTR`.

3.7. Етикети и преходи

```
@echo off
goto start

:start
echo Tova e nachaloto
goto end

:end
echo Krai
```

- `:start` и `:end` са етикети.
- `goto` прехвърля изпълнението към даден етикет.
- Това е типичен механизъм за управление на потока в Batch.

3.8. Цикли в Batch

```
@echo off
for %%i in (1 2 3 4 5) do (
    echo Chislo: %%i
)
```

- `for` повтаря дадена команда.
- Във файл се използва `%%i`, а в командния ред само `%i`.

3.9. Аргументи към Batch скрипт

```
@echo off
echo Purvi argument: %1
echo Vtori argument: %2
echo Ime na skripta: %0
```

Ако стартираме `demo.bat Sofia 11b`, тогава `%1` е `Sofia`, а `%2` е `11b`.

3.10. Пренасочване и тръби в Batch

```
dir > files.txt
type files.txt
dir | more
```

- `>` записва изхода във файл.
- `>>` добавя към края на файл.
- `|` подава изхода на една команда към друга.

3.11. Цял пример на Batch

```
@echo off
set /p user=Vuvedi потребител:
set /p folder=Vuvedi папка за syzdavane:

echo Zdravei, %user%
mkdir %folder%
echo Papkata %folder% beshe syzadadana.
pause
```

Този скрипт чете две стойности от потребителя, поздравява го и създава директория.

4. Подробен синтаксис на Bash

4.1. Основна структура на Bash скрипт

```
#!/bin/bash
# Това е коментар
echo "Nachalo na skripta"
```

- `#!/bin/bash` се нарича **shebang** и указва кой интерпретатор да изпълни файла.
- Коментарите започват със `#`.
- Текстът най-често се поставя в кавички.

4.2. Променливи в Bash

```
#!/bin/bash
name="Ivan"
echo "Zdravei, $name"
```

- Променливата се записва като `name="Ivan"`.
- Няма интервали около `=`.
- Стойността се използва с `$name` или `${name}`.

4.3. Четене на вход от потребителя

```
#!/bin/bash
read -p "Vuvedi ime: " name
echo "Zdravei, $name"
```

`read` чете въведените данни, а `-p` показва подкана.

4.4. Аритметика в Bash

```
#!/bin/bash
a=5
b=3
sum=$((a + b))
echo "Sborat e $sum"
```

В Bash често се използва конструкцията `$(( ... ))` за аритметични изрази.

4.5. Условни конструкции в Bash

```
#!/bin/bash
read -p "Vuvedi chislo: " num

if [ "$num" -ge 10 ]; then
    echo "Chisloto e po-golyamo ili ravno na 10"
else
    echo "Chisloto e po-malko ot 10"
fi
```

- `if` започва условието.
- `then` указва блока за изпълнение при вярно условие.
- `else` е незадължителен.
- `fi` завършва условната конструкция.

4.6. Сравнения в Bash

- `-eq` равно
- `-ne` различно
- `-gt` по-голямо
- `-lt` по-малко
- `-ge` по-голямо или равно
- `-le` по-малко или равно

```
if [ "$a" -eq "$b" ]; then
    echo "Ravni sa"
fi
```

4.7. Цикъл for в Bash

```
#!/bin/bash
for i in 1 2 3 4 5
do
    echo "Chislo: $i"
done
```

`done` затваря цикъла.

4.8. Цикъл while в Bash

```
#!/bin/bash
counter=1
while [ $counter -le 5 ]
do
    echo "Broqch: $counter"
    counter=$((counter + 1))
done
```

while повтаря блока, докато условието е вярно.

4.9. Аргументи към Bash скрипт

```
#!/bin/bash
echo "Purvi argument: $1"
echo "Vtori argument: $2"
echo "Broi argumenti: $#"
```

- **\$1**, **\$2** и т.н. са позиционни параметри.
- **\$#** показва броя на аргументите.
- **\$@** съдържа всички аргументи.

4.10. Функции в Bash

```
#!/bin/bash
pozdravi() {
    echo "Zdravei, $1"
}

pozdravi "Maria"
```

Функциите позволяват да групираме команди и да ги използваме многократно.

4.11. Пренасочване и тръби в Bash

```
ls > files.txt
cat files.txt
ls | wc -l
```

- **>** пренасочва изхода към файл.
- **>>** добавя към файл.
- **|** свързва командите в поредица.

4.12. Цял пример на Bash

```
#!/bin/bash
read -p "Vuvedi потребител: " user
read -p "Vuvedi papka za syzdavane: " folder

echo "Zdravei, $user"
mkdir -p "$folder"
echo "Papkata $folder beshe syzdata."

```

Този Bash скрипт изпълнява същата идея като Batch примера, но със синтаксиса на Bash.

5. Най-важните разлики в синтаксиса

Елемент	Batch	Bash
Коментар	REM komentar	# komentar
Променлива	set x=5 и %x%	x=5 и \$x
Вход от клавиатура	set /p name=Ime:	read -p "Ime: " name
Условие	if ... (...) else (...)	if ...; then ... else ... fi
Цикъл	for %%i in (...) do ...	for i in ...; do ...; done
Аргументи	%1, %2	\$1, \$2, @\$

6. Чести грешки

- Да се поставят интервали около `=` при създаване на променливи.
- Да се обърква записът на променливите: `%name%` в Batch и `$name` в Bash.
- Да се забрави `#!/bin/bash` в Bash скрипт.
- Да се стартира Bash файл без права за изпълнение.
- Да се използва синтаксис на Linux команда директно в Windows `cmd` или обратно.

7. Практически задачи

1. Създай Batch скрипт, който пита за име и извежда поздрав.
2. Създай Bash скрипт, който изчислява сумата на две числа.
3. Напиши Batch скрипт с `if`, който проверява дали число е положително.
4. Напиши Bash скрипт с `for`, който извежда числата от 1 до 10.
5. Направи и двата скрипта да приемат аргументи от командния ред.

8. Обобщение

Batch и **Bash** са езици за автоматизация, чрез които можем да записваме поредица от команди в текстов файл и да ги изпълняваме автоматично. Batch е типичен за **Windows**, а Bash - за **Linux/Unix**. Най-важното за правилната работа е да познаваме техния синтаксис: променливи, вход от потребителя, условия, цикли, аргументи и пренасочване на изхода.